Lab 23 — Data Classification and Data Loss Prevention

> **SY0-701 Objective 3.3** — Compare and contrast concepts and strategies to protect data
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~25 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Label a set of files by classification, build a content-inspection DLP scanner that finds Singapore NRIC/FIN numbers, valid card numbers and emails, catch a mislabelled file, then protect the data with masking, tokenisation, keyed hashing and permission restrictions.

You'll build: A labelled data store under `/srv/lab23`, a Python DLP scanner with a Luhn check, masked and tokenised output files, a restricted token vault, and need-to-know file permissions verified with a test user.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab needed; tools installed in step 1. **All names, NRIC/FIN numbers and card numbers are synthetic test values** (the card numbers are public payment-industry test numbers).

Background

- **Data types** include regulated data (personal data under the PDPA, cardholder data under PCI DSS), trade secrets, intellectual property, legal and financial information, in human-readable or non-human-readable form.
- **Classifications** such as Public, Private, Confidential and Restricted tell people and tools how to handle each item. Labels only help if they are correct — DLP checks that content matches its label.
- **DLP content inspection** uses patterns (regular expressions), keywords and checksums. The **Luhn check** separates real card numbers from random 16-digit strings and cuts false positives.
- **Methods to secure data: masking** hides part of a value for display; **tokenisation** swaps the value for a random token mapped in a vault; **hashing** is one-way (a keyed HMAC is needed for low-entropy IDs); **permission restrictions** enforce need to know; **segmentation** keeps the vault apart.
- Singapore's PDPC advises against collecting full NRIC numbers unless required by law — masking (for example `S****567D`) is the expected display form.

Steps

1. Install the tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y python3 openssl acl
mkdir -p /srv/lab23/{data,out,vault} && cd /srv/lab23
```

2. Create a synthetic data set

Five files with different sensitivity. One "public" press release accidentally contains a customer's NRIC.

```
cd /srv/lab23/data
cat > press-release.txt <<'EOF'
FOR IMMEDIATE RELEASE — Example Pte Ltd opens new Jurong office.
Media contact: media@example.com
Customer quote supplied by Tan Ah Kow (NRIC S1234567D) — approved for release.
EOF
cat > staff-memo.txt <<'EOF'
Internal memo: the office will close early on Friday. Questions to hr-team@example.com
EOF
cat > hr-salaries.csv <<'EOF'
name,nric_fin,email,monthly_salary
Tan Ah Kow,S1234567D,ahkow.tan@example.com,6200
Priya Raman,T0123456G,priya.r@example.com,7400
Li Wei,G1234567X,li.wei@example.com,5800
Maria Santos,M1234567K,maria.s@example.com,5100
EOF
cat > payments.csv <<'EOF'
order_id,cardholder,card_number,expiry
ORD-1001,Tan Ah Kow,4111111111111111,12/28
ORD-1002,Priya Raman,5555555555554444,03/27
ORD-1003,Li Wei,378282246310005,09/29
ORD-1004,Test Typo,4111111111111112,01/27
EOF
cat > product-formula.txt <<'EOF'
Trade secret: SuperGlue v7 cure ratio 3:1 at 42C for 90 seconds. Do not distribute.
EOF
ls -l
```

3. Label each file by classification

Record the owner's classification decision in a manifest — the label the DLP scanner will enforce.

```
cd /srv/lab23
cat > labels.csv <<'EOF'
file,classification
press-release.txt,Public
staff-memo.txt,Private
hr-salaries.csv,Confidential
payments.csv,Restricted
product-formula.txt,Confidential
EOF
column -s, -t labels.csv 2>/dev/null || cat labels.csv
```

4. Write a regex DLP scanner with a Luhn check

The scanner finds NRIC/FIN (`[STFGM]\d{7}[A-Z]`), emails and card numbers that pass the Luhn checksum, then compares what it found with what the label allows.

```
cat > /srv/lab23/dlp_scan.py <<'EOF'
#!/usr/bin/env python3
"""Minimal content-inspection DLP scanner for Lab 23 (synthetic data only).
Usage: dlp_scan.py <folder> [labels.csv]   (files without a label are treated as
Public)"""
import csv, re, sys, pathlib
```

```
NRIC = re.compile(r"\b[STFGM]\d{7}[A-Z]\b")
EMAIL = re.compile(r"\b[\w.+-]+@[\w-]+(?:\.[\w-]+)+\b")
CARD = re.compile(r"\b\d(?:[ -]?\d){12,18}\b")

def luhn_ok(candidate):
    digits = [int(c) for c in candidate if c.isdigit()]
    total = 0
    for i, d in enumerate(reversed(digits)):
        if i % 2 == 1:
            d *= 2
            if d > 9:
                d -= 9
        total += d
    return 13 <= len(digits) <= 19 and total % 10 == 0

# What each classification is allowed to contain
ALLOWED = {"Public": {"EMAIL"}, "Private": {"EMAIL"},
           "Confidential": {"EMAIL", "NRIC"}, "Restricted": {"EMAIL", "NRIC", "CARD"}}

def scan(text):
    hits = {"NRIC": NRIC.findall(text), "EMAIL": EMAIL.findall(text),
            "CARD": [c for c in CARD.findall(text) if luhn_ok(c)]}
    return {k: v for k, v in hits.items() if v}

folder = pathlib.Path(sys.argv[1])
labels = {}
if len(sys.argv) > 2:
    labels = {r["file"]: r["classification"] for r in csv.DictReader(open(sys.argv[2]))}
violations = 0
for f in sorted(p for p in folder.iterdir() if p.is_file()):
    label = labels.get(f.name, "Public")
    hits = scan(f.read_text(errors="ignore"))
    bad = set(hits) - ALLOWED[label]
    summary = ", ".join(f"{k} x{len(v)}" for k, v in hits.items()) or "no sensitive data"
    status = "VIOLATION" if bad else "ok"
    violations += bool(bad)
    print(f"[{status:9}] {f.name:24} label={label:12} found: {summary}")
    for k in sorted(bad):
        print(f"            -> {k} not permitted in {label} data: {hits[k]}")
print(f"Files with policy violations: {violations}")
sys.exit(1 if violations else 0)
EOF
chmod 755 /srv/lab23/dlp_scan.py
python3 -c "import py_compile; py_compile.compile('/srv/lab23/dlp_scan.py', doraise=True)"
&& echo "Scanner compiles"
```

5. Run the scan and find the mislabelled file

```
cd /srv/lab23
python3 dlp_scan.py data labels.csv
```

```
[ok       ] hr-salaries.csv          label=Confidential found: NRIC x4, EMAIL x4
[ok       ] payments.csv             label=Restricted   found: CARD x3
[VIOLATION] press-release.txt        label=Public       found: NRIC x1, EMAIL x1
```

```
            -> NRIC not permitted in Public data: ['S1234567D']
[ok       ] product-formula.txt      label=Confidential found: no sensitive data
[ok       ] staff-memo.txt           label=Private      found: EMAIL x1
Files with policy violations: 1
```

Only 3 of the 4 card numbers are reported: `4111111111111112` fails the Luhn check. Note that the trade secret is invisible to pattern matching — DLP relies on correct labels for that kind of data.

6. Mask and tokenise the sensitive fields

Produce a masked HR extract for display, and a tokenised payments file whose real card numbers live only in a restricted vault.

```
cat > /srv/lab23/protect.py <<'EOF'
#!/usr/bin/env python3
"""Masking and tokenisation helpers for Lab 23."""
import csv, json, secrets, sys
BASE = "/srv/lab23"
def mask_nric(v):  return v[0] + "****" + v[-4:]            # S1234567D -> S****567D
def mask_email(v): return v[0] + "***@" + v.split("@", 1)[1]
if len(sys.argv) > 2 and sys.argv[1] == "detok":
    print(json.load(open(f"{BASE}/vault/tokens.json")).get(sys.argv[2], "UNKNOWN TOKEN"))
    sys.exit()
with open(f"{BASE}/data/hr-salaries.csv") as src, open(f"{BASE}/out/hr-masked.csv", "w",
newline="") as dst:
    rows = list(csv.DictReader(src))
    w = csv.DictWriter(dst, fieldnames=rows[0].keys()); w.writeheader()
    for r in rows:
        r["nric_fin"], r["email"] = mask_nric(r["nric_fin"]), mask_email(r["email"])
        w.writerow(r)
vault = {}
with open(f"{BASE}/data/payments.csv") as src, open(f"{BASE}/out/payments-tokenised.csv",
"w", newline="") as dst:
    rows = list(csv.DictReader(src))
    w = csv.DictWriter(dst, fieldnames=rows[0].keys()); w.writeheader()
    for r in rows:
        token = "tok_" + secrets.token_hex(8)
        vault[token] = r["card_number"]
        r["card_number"] = token + "-last4-" + r["card_number"][-4:]
        w.writerow(r)
json.dump(vault, open(f"{BASE}/vault/tokens.json", "w"), indent=1)
print("Wrote out/hr-masked.csv, out/payments-tokenised.csv and vault/tokens.json")
EOF
cd /srv/lab23 && python3 protect.py
chmod 700 vault && chmod 600 vault/tokens.json
head -3 out/hr-masked.csv; head -3 out/payments-tokenised.csv
TOKEN=$(awk -F, 'NR==2{split($3,a,"-last4-"); print a[1]}' out/payments-tokenised.csv)
echo "Detokenise $TOKEN -> $(python3 protect.py detok "$TOKEN")"
```

7. Pseudonymise with a keyed hash instead of a plain hash

A plain SHA-256 of an NRIC is not anonymous: the ID space is small enough to brute force. A keyed HMAC is irreversible without the secret key.

```
cd /srv/lab23
openssl rand -hex 32 > vault/hmac.key && chmod 600 vault/hmac.key
python3 - <<'EOF'
import hashlib, hmac, itertools, string
target = hashlib.sha256(b"S1234567D").hexdigest()
print("Plain SHA-256 of the NRIC:", target[:16], "...")
# Attacker knows the first 5 digits (e.g. from birth year); tries the remaining 2 digits
and the check letter
for d1, d2, letter in itertools.product(string.digits, string.digits,
string.ascii_uppercase):
    guess = f"S12345{d1}{d2}{letter}"
    if hashlib.sha256(guess.encode()).hexdigest() == target:
        print("Brute-forced in at most 2600 guesses ->", guess)
        break
key = open("/srv/lab23/vault/hmac.key").read().strip().encode()
print("HMAC-SHA256 pseudonym   :", hmac.new(key, b"S1234567D", "sha256").hexdigest()[:16],
"... (needs the key)")
EOF
python3 - <<'EOF'
import csv, hmac
key = open("/srv/lab23/vault/hmac.key").read().strip().encode()
rows = list(csv.DictReader(open("/srv/lab23/data/hr-salaries.csv")))
with open("/srv/lab23/out/hr-pseudonymised.csv", "w", newline="") as f:
    w = csv.writer(f); w.writerow(["person_id", "monthly_salary"])
    for r in rows:
        w.writerow([hmac.new(key, r["nric_fin"].encode(), "sha256").hexdigest()[:20],
r["monthly_salary"]])
print(open("/srv/lab23/out/hr-pseudonymised.csv").read())
EOF
```

8. Apply permission restrictions by classification

Map each label to file permissions, then verify need to know with an unprivileged `analyst` account.

```
cd /srv/lab23
groupadd -f hr
id analyst >/dev/null 2>&1 || useradd -m -s /bin/bash analyst
chown root:root data/press-release.txt data/staff-memo.txt && chmod 644 data/press-
release.txt data/staff-memo.txt
chown root:hr data/hr-salaries.csv data/product-formula.txt && chmod 640 data/hr-
salaries.csv data/product-formula.txt
chown root:root data/payments.csv && chmod 600 data/payments.csv
ls -l data
su - analyst -c 'head -1 /srv/lab23/data/press-release.txt'
su - analyst -c 'cat /srv/lab23/data/hr-salaries.csv' 2>&1 | head -1
usermod -aG hr analyst
su - analyst -c 'head -2 /srv/lab23/data/hr-salaries.csv'
su - analyst -c 'cat /srv/lab23/data/payments.csv /srv/lab23/vault/tokens.json' 2>&1 |
head -2
```

```
cat: /srv/lab23/data/hr-salaries.csv: Permission denied
...after joining the hr group the analyst can read the Confidential file...
cat: /srv/lab23/data/payments.csv: Permission denied
cat: /srv/lab23/vault/tokens.json: Permission denied
```

9. Re-scan the protected output to confirm no raw PII remains

Treat everything in `out/` as Public and scan it. Masked, tokenised and pseudonymised files must pass.

```
cd /srv/lab23
python3 dlp_scan.py out && echo "Sanitised output is safe to share"
```

Test it

```
cd /srv/lab23
python3 dlp_scan.py data labels.csv | tail -1
python3 dlp_scan.py out | tail -1
su - analyst -c 'cat /srv/lab23/data/payments.csv' 2>&1 | grep -c "Permission denied"
```

Expected: `Files with policy violations: 1` (the mislabelled press release), `Files with policy violations: 0` for the protected output, and `1` (the analyst cannot read Restricted card data).

Troubleshooting

Symptom	Fix
`KeyError` in `dlp_scan.py`	A label in `labels.csv` is misspelt; use exactly Public, Private, Confidential or Restricted.
`su: user analyst does not exist`	Rerun the `useradd` line in step 8.
Step 9 reports a violation	`protect.py` did not run; check `ls out/` contains `hr-masked.csv` and `payments-tokenised.csv`.

Challenge

Fix the root cause in step 5: relabel or redact `press-release.txt`, rerun the scan to reach zero violations, then extend `dlp_scan.py` with a keyword rule (for example "Trade secret" or "Do not distribute") so the product formula is detected even if someone labels it Public.

Reflection

Your team must share salary data with an external analytics vendor. Compare masking, tokenisation and keyed hashing for this purpose: which keeps the analysis useful, which can be reversed and by whom, and where must the key or vault live?

Exam tips

- Masking = hide part of a value for display; tokenisation = random substitute with a vault mapping; encryption = reversible with a key; hashing = one-way.
- Tokenisation removes systems from PCI DSS scope because tokens have no mathematical link to the card number.
- DLP inspects content with patterns, keywords and checksums (Luhn); classification labels tell DLP which action to apply.

- Data sovereignty and geographic restrictions decide where data may be stored or accessed; permission restrictions decide who.

References

- PDPC Advisory Guidelines on the PDPA for NRIC and other national identification numbers — https://www.pdpc.gov.sg/guidelines-and-consultation
- PCI DSS v4.0 and tokenisation guidance — https://www.pcisecuritystandards.org/
- NIST SP 800-122 Guide to Protecting the Confidentiality of PII — https://csrc.nist.gov/pubs/sp/800/122/final
- OWASP Cheat Sheet: User Privacy Protection — https://cheatsheetseries.owasp.org/cheatsheets/User_Privacy_Protection_Cheat_Sheet.html

---